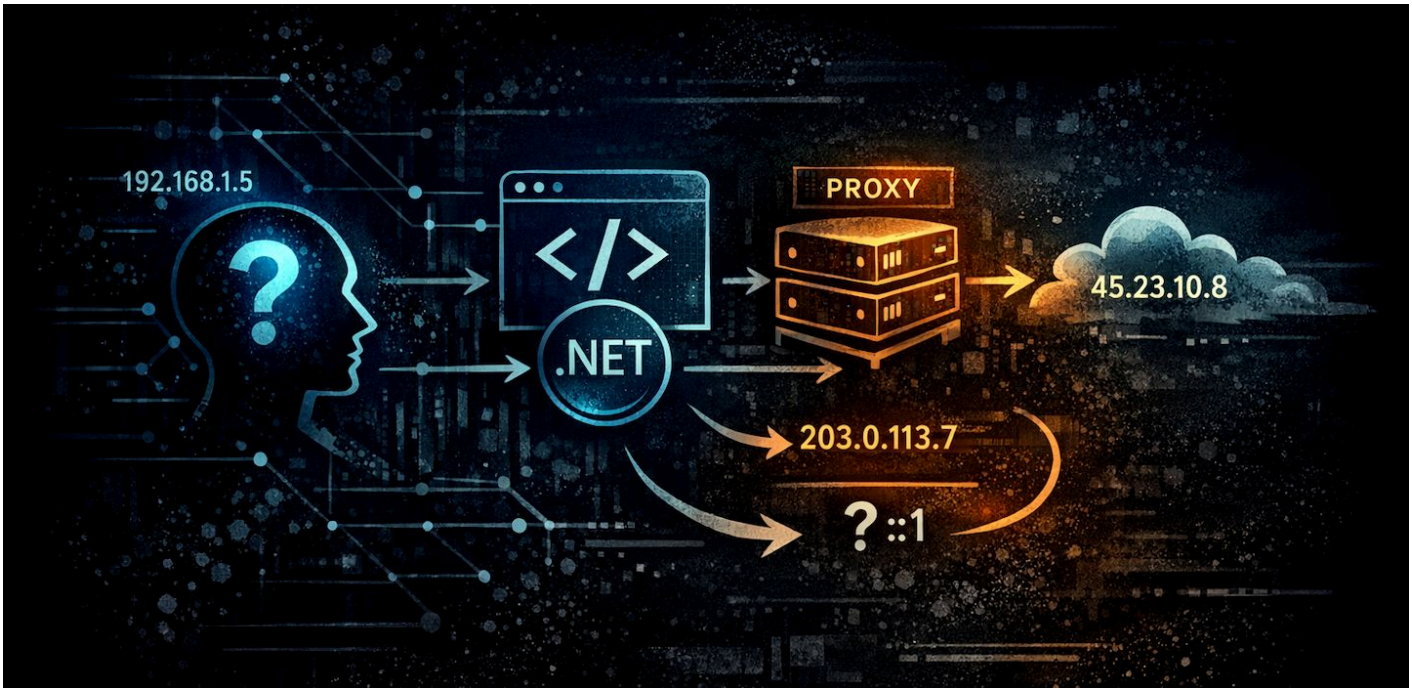
Getting the Client IP Address in ASP.NET Core



blah, blah, blah

When I need to pick up the client IP Address in ASP.NET Core I always forget where to find the connection information.

It's simple enough:

```http
POST /api/qmm/submit-item HTTP/2.0
Content-Type: application/json
Accept: text/xml, application/json
Accept-Encoding: gzip, deflate, br
WebSurge-Request-InjectJsonBearerToken: token
Authorization: Bearer asddasddasd
Websurge-Request-Name: Submit a new Item

{
  "QueueName": "Test1",
  "Message": "Test Submission",
  "Action": "PRINT",
  "Xml": "<doc><value>Hello</value></doc>"
}
```

```csharp
int x = 1;
x++;
HttpContext?.Connection?.RemoteIpAddress
```

```razor
<textarea asp-for="Product.SpecialsText"
                class="form-control mt-2" style="height: 6.3em"
                placeholder="Detailed specials text (optional)"></textarea>

<hr/>

<button type="submit" name="btnSubmit" id="btnSubmit"
        class="btn btn-primary btn-lg" accesskey="S">
    <i class="fas fa-check text-success"></i>
    Save Product
</button>

<div class="float-end">
    <button type="submit" name="btnDeleteProduct" id="btnDeleteProduct"
            title="Delete this product"
            onclick="if(!confirm('Are you sure you want to delete this item?\n\n
@Model.Product.Sku - @Model.Product.Description')) return false;"
            class="btn btn-secondary btn-sm">
        <i class="fas fa-times text-danger"></i>
        Delete
    </button>

    <a href="/admin/productmanager"
       class="btn btn-secondary btn-sm">
        <i class="fas fa-times text-danger"></i>
        Cancel
    </a>
</div>

@{
    var scriptVars = new ScriptVariables("globals");
    scriptVars.Add("sku", Model.Product.Sku);
}

@scriptVars.ToHtmlString(true)

<input type="hidden" asp-for="Product.Id" />
<input type="hidden" asp-for="PreviousListUrl" />

</form>
```

but I never remember to look on the context *object* as I expect it to be on the Request 😄.

It's also useful to remember that if requests are proxied, we need to return the **forwarded IP address**, rather than the proxy's IP Address. Finally, in most cases you'd likely want the ipv4 address rather than an IPv6 address.

##AD##

Here's ready to use helper extension method for the `HttpRequest` class that makes this more easily accessible:

csharp

```csharp
/// <summary>
/// Returns the client IPv4 Address for a request.
///
/// Checks proxy forwarding first, the actual ip
/// and returns null.
/// </summary>
/// <param name="request">The HttpRequest instance.</param>
/// <param name="checkForProxy">
/// Indicates whether to check for proxy headers.
///
/// Default returns the un-translated connection's IP Address
/// returned by the Web server.
///
/// When true, checks the Proxy forwarding headers
/// `X-Forwarded-For`, `Forwarded` and `X-Real-IP`
/// in that order and returns the 1st valid IP address found.
/// </param>
/// <returns>IP Address or null</returns>
public static string GetClientIpAddress(this HttpRequest request, bool
checkForProxy = false)
{
    if (request == null) return null;

    string ip =
NormalizeIpAddress(request.HttpContext?.Connection?.RemoteIpAddress);
    if (!checkForProxy)
        return ip;

    string proxy = GetForwardedIpAddress(request.Headers["X-Forwarded-
For"].FirstOrDefault());
    if (!string.IsNullOrEmpty(proxy))
        return proxy;

    proxy = GetForwardedIpAddress(request.Headers["Forwarded"].FirstOrDefault(),
true);
    if (!string.IsNullOrEmpty(proxy))
        return proxy;

    proxy = GetForwardedIpAddress(request.Headers["X-Real-
IP"].FirstOrDefault());
    return proxy ?? ip;
}
```

```
/// <summary>
/// Handle various forwarding headers and their custom parsing
/// of multiple proxy chain values
/// </summary>
/// <param name="headerValue">The value of the forwarding header.</param>
/// <param name="isForwardedHeader">Indicates if the header is the `Forwarded`
header.</param>
/// <returns>The extracted IP address or null if none found.</returns>
private static string GetForwardedIpAddress(string headerValue, bool
isForwardedHeader = false)
{
    if (string.IsNullOrWhiteSpace(headerValue))
        return null;

    foreach (var value in headerValue.Split(','))
    {
        string candidate = value?.Trim();
        if (string.IsNullOrWhiteSpace(candidate))
            continue;

        if (isForwardedHeader)
        {
            candidate = candidate.Split(';')
                .Select(segment => segment?.Trim())
                .FirstOrDefault(segment => segment != null &&
segment.StartsWith("for=", StringComparison.OrdinalIgnoreCase));

            if (string.IsNullOrWhiteSpace(candidate))
                continue;

            candidate = candidate.Substring(4).Trim();
        }

        candidate = candidate.Trim('"');

        if (candidate.StartsWith("[", StringComparison.Ordinal) &&
candidate.Contains("]", StringComparison.Ordinal))
            candidate = candidate.Substring(1, candidate.IndexOf(']') - 1);
        else if (candidate.Count(ch => ch == ':') == 1)
        {
            var parts = candidate.Split(':');
            if (parts.Length == 2 && IPAddress.TryParse(parts[0], out _))
                candidate = parts[0];
        }
```

```
        if (string.Equals(candidate, "unknown",
StringComparison.OrdinalIgnoreCase))
            continue;

        if (IPAddress.TryParse(candidate, out var address))
            return NormalizeIpAddress(address);
    }

    return null;
}

/// <summary>
/// Return as IPv4 address if the address is an IPv4-mapped IPv6 address,
/// otherwise return the original address as a string.
/// </summary>
/// <param name="address"></param>
/// <returns></returns>
private static string NormalizeIpAddress(IPAddress address)
{
    if (address == null)
        return null;

    if (address.IsIPv4MappedToIPv6)
        address = address.MapToIPv4();

    return address.ToString();
}
```

The bulk of this code is related to the Proxy forwarding handling which is optional. If you know you're directly connected to the Internet, you can skip the proxy forwarding stuff - in fact it's a good idea to do this to avoid any spoofing from a client. The various forwarding headers provide multiple IP Addresses in the proxy chain and you essentially need to pick out the first IP address to get the original address.

You can also find this as part of the `HttpRequestExtensions` class in the `Westwind.AspNetCore` package here which provides a host of other small, but frequently used extensions.

Alternative: Use the IP Forwarded Headers Middleware

thanks to @RichardD in the comments

If you know you are always running behind a proxy server, and you need the IP Address in all or most requests, you can run the Forwarded Headers Middleware which handles the above logic and simply populates the `HttpContext.Connection.RemoteIpAddress` making the process complete transparent. That certainly works, but depending on how you use IP Address might be overkill.

The middleware is configured like this in the service configuration during startup:

```csharp
builder.Services.Configure<ForwardedHeadersOptions>(options =>
{
    options.ForwardLimit = 2;
    options.KnownProxies.Add(IPAddress.Parse("127.0.10.1"));
    options.ForwardedForHeaderName = "X-Forwarded-For-My-Custom-Header-Name";
});

...

// near the very top of the middleware pipeline
// to ensure subsequent middleware pieces get the updated addresses
app.UseForwardedHeaders();
```

There's more info on the Microsoft site on how the middleware works here.

##AD##

Summary

Nothing new here, but given how often I fumble around with this value, creating a wrapper and putting a reminder here for quick lookup seems worth the effort 😄

Resources

- Westwind.AspNetCore on Github
- Configure ASP.NET Core to work with proxy servers and load balancers

this post was created and published with the **Markdown Monster Editor**